확인 시험 풀이

시험지의 실행 예는 아래 풀이 코드를 실제로 실행한 결과다

유형 A 입력과 정렬

1. 임의 개수의 정수를 입력하여, 같은 수는 한 번만 남기고 오름차순으로 소팅하는 프로그램을 작성하라. 맨 마지막에 0을 넣어 입력의 끝을 가리키도록 하라.

풀이 핵심

- 0을 읽으면 break. 0은 배열에 넣지 않는다.
- 넣기 전에 배열을 훑어 같은 값이 있으면 넣지 않는다.
- 버블 정렬로 오름차순 정렬한 뒤 출력한다.

풀이 코드

```c
/* 1번: 같은 수를 빼고 오름차순 정렬 */
#include <stdio.h>
#define MAX 1000

int main(void) {
    int a[MAX], n = 0, x, i, j, tmp, dup;

    printf("Enter integers>> ");
    while (n < MAX && scanf("%d", &x) == 1) {
        if (x == 0) break;                  /* 0은 입력의 끝 */
        dup = 0;
        for (i = 0; i < n; i++)             /* 이미 있는 값인지 확인 */
            if (a[i] == x) { dup = 1; break; }
        if (!dup) a[n++] = x;
    }
    for (i = 0; i < n - 1; i++)             /* 버블 정렬 */
        for (j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j + 1]) {
                tmp = a[j]; a[j] = a[j + 1]; a[j + 1] = tmp;
            }
    printf("After sorting:");
    for (i = 0; i < n; i++) printf(" %d", a[i]);
    printf("\n");
    return 0;
}
```

유형 B 소수와 약수

2. 두 정수를 입력하여, 두 수 사이(두 수 포함)에 있는 소수를 모두 출력하고, 소수의 개수를 출력하는 프로그램을 작성하라. 작은 수를 먼저 입력하지 않아도 된다. 두 정수가 모두 0이면 프로그램을 종료하라.

풀이 핵심

- 입력 순서와 상관없이 작은 수 lo, 큰 수 hi를 정한다.
- is_prime 함수: 2부터 d * d <= x인 동안 나누어 본다. 1 이하는 소수가 아니다.
- 소수를 찾을 때마다 출력하고 개수를 센다.

풀이 코드

```c
/* 2번: 두 수 사이의 소수 */
#include <stdio.h>

int is_prime(int x) {
    int d;
    if (x < 2) return 0;
    for (d = 2; d * d <= x; d++)
        if (x % d == 0) return 0;
    return 1;
}

int main(void) {
    int a, b, lo, hi, i, cnt;

    while (1) {
        printf("Enter two integers>> ");
        if (scanf("%d %d", &a, &b) != 2) break;
        if (a == 0 && b == 0) break;

        lo = (a < b) ? a : b;              /* 작은 수부터 큰 수까지 */
        hi = (a < b) ? b : a;
        cnt = 0;
        printf(" ");
        for (i = lo; i <= hi; i++)
            if (is_prime(i)) {
                printf(" %d", i);
                cnt++;
            }
        printf("\n  Count: %d\n", cnt);
    }
    return 0;
}
```

유형 C 문자열

3. 입력 스트링들을 읽어서, 문장 안에 있는 단어들을 길이가 짧은 순서로 정렬하여 출력하라. 길이가 같은 단어는 알파벳 순서로 출력한다. "exit"를 입력하면 프로그램을 종료하라.

풀이 핵심

- fgets로 한 줄을 읽고, strtok으로 공백마다 잘라 words에 복사한다.
- 버블 정렬의 비교만 바꾼다: 앞 단어가 더 길거나, 길이가 같고 strcmp가 0보다 크면 바꾼다.
- 교환은 strcpy 세 번(tmp 사용).

풀이 코드

```c
/* 3번: 단어를 길이 순으로 정렬 (길이가 같으면 사전 순) */
#include <stdio.h>
#include <string.h>
#define MAX_WORDS 100
#define MAX_LEN 50

int main(void) {
    char line[1000], words[MAX_WORDS][MAX_LEN], tmp[MAX_LEN], *tok;
    int n, i, j, la, lb;

    while (1) {
        printf("Enter strings>> ");
        if (fgets(line, sizeof(line), stdin) == NULL) break;
        line[strcspn(line, "\n")] = '\0';
        if (strcmp(line, "exit") == 0) break;

        n = 0;
        tok = strtok(line, " ");
        while (tok != NULL && n < MAX_WORDS) {
            strcpy(words[n++], tok);
            tok = strtok(NULL, " ");
        }
        for (i = 0; i < n - 1; i++)
            for (j = 0; j < n - 1 - i; j++) {
                la = strlen(words[j]);
                lb = strlen(words[j + 1]);
                /* 더 길거나, 길이가 같고 사전 순으로 뒤이면 바꾼다 */
                if (la > lb || (la == lb && strcmp(words[j], words[j + 1]) > 0)) {
                    strcpy(tmp, words[j]);
                    strcpy(words[j], words[j + 1]);
                    strcpy(words[j + 1], tmp);
                }
            }
        printf(" ");
        for (i = 0; i < n; i++) printf(" %s", words[i]);
        printf("\n");
    }
    return 0;
}
```

유형 D 스택과 수식

4. Postfix notation으로 작성된 수식을 입력으로 받아서 계산한 결과를 출력하는 프로그램을 작성하라. 수식은 양의 정수와 사칙연산자(+, -, *, /)로 구성되며 공백으로 구분된다. 올바르지 않은 수식이면 Error를 출력한다. "quit"이 입력되면 프로그램을 종료한다.

풀이 핵심

- 숫자는 끝까지 읽어 스택에 넣는다.
- 연산자를 만나면 두 값을 꺼낸다. 먼저 꺼낸 값이 오른쪽(b), 나중에 꺼낸 값이 왼쪽(a)이다. 꺼낼 값이 둘 없으면 Error.
- 끝에 스택에 값이 정확히 하나 남아야 답이다. 둘 이상 남으면 Error.

풀이 코드

```c
/* 4번: 후위식 계산 */
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(void) {
    char line[200];
    int st[100], top, i, num, a, b, ok;

    while (1) {
        printf("Enter a postfix expression>> ");
        if (fgets(line, sizeof(line), stdin) == NULL) break;
        line[strcspn(line, "\n")] = '\0';
        if (strcmp(line, "quit") == 0) break;

        top = -1;
        ok = 1;
        i = 0;
        while (line[i] != '\0') {
            if (isdigit(line[i])) {               /* 숫자는 끝까지 읽어 넣는다 */
                num = 0;
                while (isdigit(line[i]))
                    num = num * 10 + (line[i++] - '0');
                st[++top] = num;
                continue;
            }
            if (strchr("+-*/", line[i]) != NULL) {
                if (top < 1) { ok = 0; break; }   /* 꺼낼 값이 둘 없다 */
                b = st[top--];                    /* 먼저 꺼낸 값이 오른쪽 */
                a = st[top--];
                switch (line[i]) {
                    case '+': st[++top] = a + b; break;
                    case '-': st[++top] = a - b; break;
                    case '*': st[++top] = a * b; break;
                    case '/': st[++top] = a / b; break;
                }
            }
            i++;
        }
        if (ok && top == 0) printf("  Value: %d\n", st[top]);
        else printf("  Error\n");
    }
    return 0;
}
```

유형 E 숫자 표기

5. 주어진 정수를 세 자리마다 쉼표(,)를 넣어 표기하는 프로그램을 작성하라. 음수도 입력할 수 있다. 0을 입력하면 종료하라.

풀이 핵심

- 음수이면 -를 먼저 출력하고 양수로 바꾼다.
- unit을 1000씩 곱해 맨 앞 묶음의 자리를 찾는다. (1234567이면 unit = 1000000)
- 맨 앞 묶음은 그대로, 나머지 묶음은 %03lld로 0을 채워 출력한다. 1002003에서 002, 003이 그렇다.

풀이 코드

```c
/* 5번: 세 자리마다 쉼표 */
#include <stdio.h>

int main(void) {
    long long n, unit;
    int first;

    while (1) {
        printf("Enter an integer>> ");
        if (scanf("%lld", &n) != 1 || n == 0) break;

        if (n < 0) { printf("  -"); n = -n; }
        else printf("  ");
        unit = 1;
        while (unit * 1000 <= n) unit *= 1000;   /* 맨 앞 묶음의 자리 */
        first = 1;
        while (unit >= 1) {
            if (first) printf("%lld", n / unit);         /* 맨 앞은 0을 채우지 않는다 */
            else printf(",%03lld", n / unit);            /* 나머지는 세 자리를 0으로 채운다 */
            n %= unit;
            unit /= 1000;
            first = 0;
        }
        printf("\n");
    }
    return 0;
}
```

유형 F 그 밖

6. 연결리스트를 이용하여, 입력한 양의 정수들을 입력한 순서대로 리스트의 끝에 붙여라. 데이터의 끝에 음수를 넣는 방법을 이용하라. 리스트를 출력한 뒤, 리스트를 뒤집어서 다시 출력하라. 첫 번째 정수가 음수이면 실행을 종료하라.

풀이 핵심

- tail 포인터를 두고 새 노드를 tail->next에 붙인다. 첫 노드면 head에 둔다.
- 뒤집기: prev = NULL, cur = head에서 시작해 next를 먼저 저장하고 cur->next = prev로 방향을 바꾼다.
- 끝나면 head = prev. 다 쓴 노드는 free한다.

풀이 코드

```c
/* 6번: 연결리스트를 뒤집기 */
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    int data;
    struct node *next;
} Node;

void print_list(const char *title, Node *head) {
    Node *p;
    printf("  %s:", title);
    for (p = head; p != NULL; p = p->next) printf(" %d", p->data);
    printf("\n");
}

int main(void) {
    Node *head, *tail, *n, *prev, *cur, *next;
    int x;

    while (1) {
        printf("Enter integers>> ");
        if (scanf("%d", &x) != 1 || x < 0) break;

        head = tail = NULL;
        while (x >= 0) {                        /* 입력 순서대로 끝에 붙인다 */
            n = (Node *)malloc(sizeof(Node));
            n->data = x;
            n->next = NULL;
            if (head == NULL) head = n;
            else tail->next = n;
            tail = n;
            if (scanf("%d", &x) != 1) break;
        }
        print_list("List", head);

        prev = NULL;                            /* 화살표 방향을 하나씩 바꾼다 */
        cur = head;
        while (cur != NULL) {
            next = cur->next;
            cur->next = prev;
            prev = cur;
            cur = next;
        }
        head = prev;
        print_list("Reversed", head);

        while (head != NULL) { n = head->next; free(head); head = n; }
    }
    return 0;
}
```